

Object Following Robot using Arduino Uno



MINI-PROJECT PRESENTATION

Department of Electronics and Communication Engineering
Cooch Behar Government Engineering College
Maulana Abul Kalam Azad University of Technology (MAKAUT)

PROJECT MEMBERS

Bikkram Das (34900323012) | Biraj Sarkar (34900323013) | Debanjan Chakraborty (34900323015)
| Sayan Sarkar (34900323048) | Soumyajit Pandit (34900323052)

PROJECT SUPERVISOR Dr. Gautam Das

01. Introduction & Overview

Motivation, HMI background, sensor envelope

02. System Architecture

Block diagram & signal flow

03. Hardware & Sensors

Arduino Uno, HC-SR04, IR modules

04. Actuators & Motor Drivers

BO geared motors, L298N H-Bridge

05. Power Scheme & Stability

Split-rail power, brownout fix

06. Software Control Algorithm

State machine & decision logic

07. Firmware Source Code

Arduino C++ with NewPing library

08. Calibration & Integration

Thresholds, chassis assembly

09. Performance Testing

Accuracy, latency, runtime

10. Conclusions

Outcomes & design validation

11. Future Scope

ESP32-CAM, PID, active scanning

Introduction & Project Overview

Autonomous target following

Following is a core capability of Human–Machine Interaction (HMI) — enabling hands-free operation and natural cooperation between people and mobile platforms.

Why not a camera?

Vision-based tracking (e.g. Raspberry Pi + camera) needs heavy processing, more weight, cost and power. Acoustic and IR sensors are a low-cost, low-power alternative.

Multi-sensor configuration

A front HC-SR04 ultrasonic sensor plus two side-mounted IR sensors together define a clean spatial envelope for real-time following.

Real-world applications

- Industrial automation carts
- Hospital carts following staff
- Hands-free warehouse transport
- Automated shopping-guide robots

Design goal:

low-cost | **low-power** | **real-time**

System Architecture

Sensor input

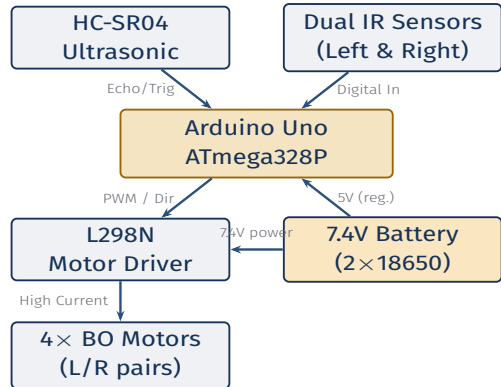
HC-SR04 gives front distance; left/right IR modules flag lateral offset.

Central processing

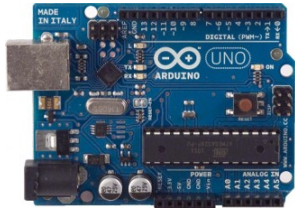
Arduino Uno runs a state machine that turns sensor readings into direction and PWM speed commands.

Actuation

L298N H-bridge drives four BO geared motors; steering is by differential wheel speed.



Hardware: Arduino Uno & Sensors



Arduino Uno — ATmega328P, 16 MHz, 14 digital I/O
(6 PWM), 6 analog in



HC-SR04 ultrasonic sensor (left) and IR proximity
module (right)

Arduino Uno board

Runs the control loop that reads every sensor and regulates the actuators each cycle.

HC-SR04 ultrasonic sensor

Emits 40 kHz pulses; distance from echo time:

$$d = \frac{t \times 0.034}{2} \text{ cm}$$

Range 2–400 cm, accuracy ± 3 mm.

IR proximity sensors

Dual side-mounted units emit IR light; a reflection lowers photodiode resistance, driving the output LOW when a target is within 8 cm.

Actuators & Motor Driver

L298N H-bridge driver

Switches high-power transistors to control DC-motor speed and direction; up to 2 A per channel. Average motor voltage is set by PWM on the Enable pins (ENA, ENB).

Brushed DC BO geared motors

Four motors, 3–9 V operating range, 1:48 gear reduction
→ 150 RPM at 6 V with high torque.

4WD kinematics

Motors wired as parallel left/right pairs. Differential speed between the pairs gives forward drive, reverse, and pivot turns.



L298N H-bridge module



BO geared DC motor

Power Scheme & System Stability

The problem

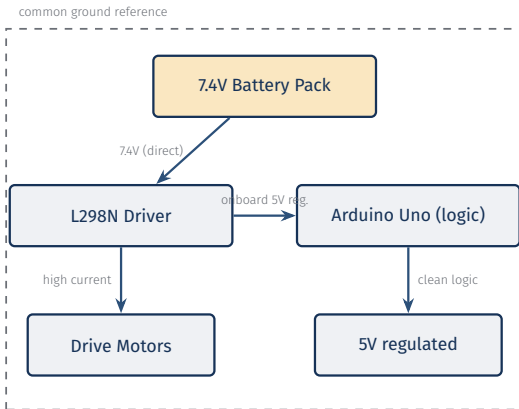
High DC-motor starting current pulls battery voltage down, resetting the microcontroller. An earlier prototype (with a servo + OLED) browned out during active scans.

The fix: split power rails

The 7.4V pack feeds the L298N directly; the driver's onboard 5V regulator powers the Arduino separately, so motor current spikes never touch the logic rail. All grounds are tied together for a common reference.

Outcome

Servo and OLED were dropped from the final build; the rail split alone gave stable, continuous operation.



Isolation prevents motor-current spikes from resetting the MCU.

Software Control Algorithm

Control framework

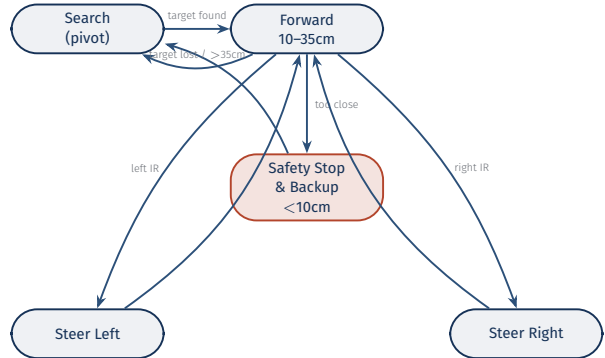
Written in C++ (Arduino IDE 2.x). The NewPing library gives non-blocking acoustic ranging while digital pins are polled for IR state.

Closed-loop transitions

Distance and lateral IR states are continually re-evaluated, triggering one of five direction codes each cycle.

Speed control

Follow envelope: 10–35 cm. PWM is modulated (170/220) based on distance to slow down as the target gets close.



Control Firmware — Source Code

Core `loop()` structure

Reads ultrasonic distance
and both IR flanks each cycle
to pick a motor action.

Safety backup & turn

Target closer than 10 cm →
halt, reverse, turn right, halt
— then re-evaluate.

Rotating search

Target lost or beyond 35 cm
→ robot pivots right to
re-acquire.

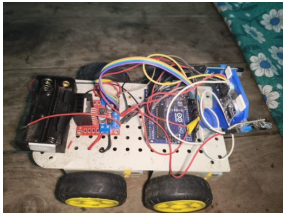
```
1 void loop() {  
2   int distance = sonar.ping_cm();  
3   if (distance == 0) distance = 200;  
4   int rightIR = digitalRead(RIGHT_IR);  
5   int leftIR  = digitalRead(LEFT_IR);  
6   analogWrite(ENA, (distance > 25) ? 220 : 170);  
7   analogWrite(ENB, (distance > 25) ? 220 : 170);  
8  
9   if (distance < MIN_DISTANCE) {           // Safety Stop & Backup  
10    setMotors('S'); delay(300);  
11    setMotors('B'); delay(400);  
12    setMotors('S'); delay(200);  
13    setMotors('R'); delay(300);  
14    setMotors('S'); return;  
15  }  
16  if (rightIR == LOW && leftIR == HIGH) setMotors('R');  
17  else if (rightIR == HIGH && leftIR == LOW) setMotors('L');  
18  else if (distance <= FOLLOW_DISTANCE) setMotors('F');  
19  else setMotors('R');                  // Search Mode  
20 }
```

Sensor threshold configuration

IR sensors are trimmed via onboard potentiometers to ignore ambient light and trigger reliably at a maximum range of 8 cm.

Physical integration

All modules mount on a 4-wheel acrylic chassis; a common ground plane removes electrical noise and signal fluctuation.



Command-state calibration map

Condition	Action
Distance < 10 cm	Safety backup & turn
10 cm – 35 cm	Drive forward
Left IR triggered	Pivot left
Right IR triggered	Pivot right
Distance > 35 cm	Target lost → search

Performance Testing & Evaluation

± 0.5 cm

ranging variance

10 ms

control loop latency

50 min

continuous runtime

Target tracking

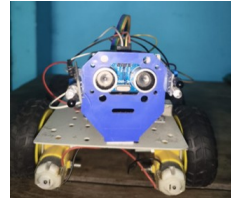
Maintains the 10–35 cm follow boundary reliably, no manual input.

Ranging & latency

10 ms loop latency enables real-time reaction to target speed changes.

Power runtime

Dual 18650 cells sustain full motor load for 50 continuous minutes.



Conclusions

Successful prototype

Built a low-cost, low-power autonomous robot that tracks and follows a target in real time, validating the core design.

Verified differential steering

Differential drive combined with state-command sensor feedback handles lateral and forward movement cleanly.

Coordinated sensor control

Simple acoustic ranging plus IR proximity gives a reliable tracking envelope — no expensive vision hardware required.

Practical experience

Hands-on exposure to hardware–software co-design, power isolation, sensor calibration, and closed-loop control.

A complete, working object-following platform built entirely on low-cost, readily available components.

Future Scope

Active scanning servo

Re-integrate an SG90 micro servo to sweep the ultrasonic sensor across a 180° field of view.

Onboard OLED display

Re-introduce a 0.96" SSD1306 screen to show live distance and operating status.

LM2596 power isolation

Add a step-down buck converter to isolate servo current spikes from the logic rail, removing the earlier reset issue for good.

ESP32-CAM recognition

Upgrade the controller to an ESP32 with onboard camera for face detection and human recognition.

PID control steering

Add a Proportional–Integral–Derivative loop for smoother acceleration and turning, replacing today's fixed-speed steps.

Thank You

Questions & Discussion